

HEAT Accessibility Extensions: Extending a Haskell Educational IDE for Visually Impaired Learners

Yarushah (ys321)
School of Computing
University of Kent
Canterbury, UK

Goutham (gs100)
School of Computing
University of Kent
Canterbury, UK

Thabitha (ta106)
School of Computing
University of Kent
Canterbury, UK

Rinku Kumari (rk105)
School of Computing
University of Kent
Canterbury, UK

Abstract—HEAT (Haskell Educational Advancement Tool) is a Java Swing IDE for undergraduate Haskell learners that was not designed with accessibility in mind. This report documents the work of Group 5 to extend HEAT for students with visual disabilities, as required by the customer brief from the module convenor. We identified four barriers in the original tool—absence of audio feedback, reliance on colour-coded console output, no accessible font size control, and mouse-dependent file navigation—and built five features to address them: Text-to-Speech (TTS) with plain English error simplification, a Deuteranopia colour blind mode, a keyboard-driven font size control with `Cmd+=` and `Cmd+-` shortcuts, an accessible startup interface that provides keyboard and TTS access before the main window appears, and three keyboard-only file dialogs. Of 56 acceptance criteria, 50 were satisfied (approximately 89%). We report honestly on limitations, including the absence of testing with visually impaired participants, and outline prioritised future work.

Keywords—accessibility, Text-to-Speech, colour vision deficiency, keyboard navigation, Java Swing, software engineering, HCI, Haskell IDE

I. INTRODUCTION

HEAT [1] is a Java Swing IDE for undergraduate Haskell learners [1]. It has gone through several development iterations: earlier versions used the Hugs interpreter, and the current version uses GHCi. The console window was deliberately designed with a colour-coded output scheme modelled on WinHugs: the GHCi input prompt is rendered in green and compiler errors in red. This colour scheme was a conscious usability decision to make the two types of output visually distinct for sighted users. The console is built on a `JTextPane` with styled text support, which is the same architecture our colour accessibility work hooks into.

HEAT was built for general educational use and was not designed with accessibility in mind. A blind student, a student with Deuteranopia, or a keyboard-only user all hit barriers in the current tool that are not the result of poor design—they are the result of accessibility not being a requirement when those design decisions were made.

This project extended HEAT to support students with visual disabilities. The client, Rogério de Lemos (module convenor), confirmed accessibility for visually impaired users

as the project focus and explicitly recommended a colour blindness feature covering at least one specific category. We identified four barriers in the original HEAT and built features to address each one:

- 1) No audio feedback—a blind user cannot tell whether compilation succeeded or failed.
- 2) Colour-coded output only—the green and red console colours that distinguish prompt from error are indistinguishable under Deuteranopia.
- 3) No accessible font size control—partially sighted users cannot increase text size without editing config files.
- 4) Mouse-dependent file navigation—keyboard-only users cannot open or create files through HEAT’s standard dialogs, and have no keyboard entry point into the application at all on first launch.

The features built to address these are: Text-to-Speech (TTS) with plain English error simplification, a Deuteranopia colour blind mode, a font size control panel with keyboard shortcuts (`Cmd+=` and `Cmd+-`), an accessible startup interface that gives keyboard and TTS access before the main window appears, and three keyboard-driven file dialogs. A project consultant also provided mid-week feedback: TTS should speak all GHCi error lines including detail lines, and HEAT should launch maximised. Both requirements were implemented.

II. GROUP ORGANISATION

Table I shows feature ownership as confirmed at Meeting 2 on 2 Jun 2026. Rinku Kumari was assigned settings integration, text enlargement, and icon enlargement. Her code was not in a working state by the deadline and could not be integrated into the final submission. Thabitha took over text enlargement and icon enlargement as additional contributions. Settings integration was absorbed into the final merge by Yarushah.

III. GROUP ORGANISATION AND COMMUNICATION

A. Development Methodology

The team adopted a Kanban development approach rather than Scrum or RUP. Work items were tracked through Not

Table I
FEATURE OWNERSHIP

Feature	Owner
TTS and error simplification	Yarushah
Keyboard navigation and shortcuts	Yarushah
Keyboard Guide (F1)	Yarushah
Colour blind mode (Deuteranopia)	Thabitha
Font size control	Thabitha
Icon enlargement	Thabitha
Splash screen	Goutham
Keyboard file dialogs (3)	Goutham
Settings integration	Rinku (not delivered)

Started, In Progress, Testing, and Done states via Microsoft Teams. Daily informal stand-ups replaced formal sprint ceremonies. The project ran for nine days—too short for sprint planning overhead to add value.

B. Synchronous Meetings

Six formal meetings were held over the nine-day period. The kickoff session assigned feature ownership and established the Git branching strategy. A midway meeting reviewed progress and evaluated the status of each member’s contributions. An online meeting involving Yarushah, Thabitha, and Goutham covered colour accessibility while Rinku was unavailable. A subsequent integration session resolved merge conflicts. A final online meeting coordinated testing. The last in-person meeting covered system integration and report preparation. Total synchronous collaboration was approximately 5.5 hours.

C. Asynchronous Communication

Daily stand-up updates were posted each morning via Teams, covering completed work, planned activities, and blockers. Voice calls were used for complex issues such as merge conflicts and integration challenges.

D. Pair Programming

Key pair programming sessions addressed the thread-safe `BlockingQueue` for TTS (Yarushah and Goutham), colour accessibility integration with the console interface (Thabitha and Yarushah), and keyboard file dialog search logic (Goutham and Yarushah).

E. Branch Management and Code Review

Feature branches were maintained for TTS, keyboard navigation, colour accessibility, font size, splash screen, and keyboard dialogs. Final merges required review by at least one additional team member.

A significant challenge occurred when Rinku committed incomplete code directly to the master branch without prior review, resulting in a broken build. The team responded by restoring a stable baseline and strengthening review requirements. As a consequence the team stopped relying on GitLab

for sharing work in progress and instead shared files locally, working independently and only pushing once features were complete. This is why the final integration involved manually combining three separate codebases rather than incremental branch merges.

F. Project Metrics

Over the nine-day period: six formal meetings, approximately 5.5 hours synchronous and over 50 hours asynchronous collaboration. Five major accessibility enhancements were delivered. One planned feature (settings integration, assigned to Rinku) was not delivered and was absorbed by Yarushah. Overall, 50 of 56 acceptance criteria were satisfied (89%).

IV. REQUIREMENTS

A. User Base Justification

The user base was specified by the customer brief: the module convenor required HEAT to be extended to support students with visual disabilities. For a UK university tool the Equality Act 2010 and the Public Sector Bodies Accessibility Regulations 2018 establish an anticipatory duty to make reasonable adjustments; WCAG 2.1 is the commonly referenced technical standard. Visually impaired programmers and learners are a documented population validated by prior HCI research (CodeTalk—Potluri et al., CHI 2018; StructJumper—Baker et al., CHI 2015). Universal-design benefits also apply: features designed for the most constrained users improve usability for low-vision users and sighted power users.

Honest caveat: we did not perform formal user testing with visually impaired participants. Our validation is by customer requirement, design principle, standards alignment, and literature precedent rather than by original user trials.

B. Problem Analysis

Blind users rely entirely on audio—HEAT provides none. Every status change, error, and result is visual only. *Deuteranopia users* cannot distinguish red from green; under Coblis [6] simulation the default red errors and green prompt are visually identical. *Partially sighted users* need larger text but cannot navigate the Options menu to get it if they cannot read the current size.

C. Approaches Considered

For TTS: FreeTTS [2], Google Cloud TTS, and OS-native speech were evaluated. OS-native was chosen: no external dependencies, offline operation, works on macOS (`say`), Windows (PowerShell `SpeechSynthesizer`), and Linux (`espeak`).

For colour blind mode: targeted console colour substitution was preferred over full palette replacement. The console’s `JTextPane` applies colour per character run via

Table II
SAMPLE ACCEPTANCE CRITERIA (FULL SET IN WIKI)

ID	Given	Then	Status
AC-02	TTS on, file open	Speaks success	Pass
AC-08	TTS on, type error	Plain English spoken	Pass
AC-12	TTS disabled	HEAT is silent	Pass
AC-23	Deuteranopia on	Errors render orange	Pass
AC-35	At max 36pt	A+ disabled	Pass
AC-36	Cmd+= pressed	Font size changes	Pass
AC-42	Splash + Cmd+N	TTS confirms, dialog opens	Pass
AC-52	Settings saved	Preferences restored	Not tested

StyledDocument, making targeted substitution sufficient and low-risk.

For font size: an always-visible panel with keyboard shortcuts over a settings-only approach. A user who cannot read the current font size cannot reliably navigate menus to change it.

For file navigation: search-based dialogs over JFileChooser browsing. A blind user listening to every filename is slow [4]; typing a name fragment and hearing only matching results is much faster.

D. User Stories and Acceptance Criteria

Thirty-two user stories cover five feature areas (US-01–US-32). Table II shows a representative sample. Fifty-six acceptance criteria were defined; 50 pass. AC-28 (responsiveness after Apply in Options) and AC-36 (font size keyboard shortcut in editor context) are recorded as known limitations. AC-52–AC-56 (Rinku’s undelivered features) are marked not tested. AC-12 was tested by setting `TTS_ENABLED=false` in `heat.settings`: HEAT confirmed completely silent. Status: Pass.

V. DESIGN

A. HCI Rationale

The hardest design problem was file navigation. Java’s built-in JFileChooser shows a grid of folders and files that you navigate by clicking—essentially unusable for a blind user. Our approach was to replace browsing with searching: the user types a name fragment and receives only matching files. This fits what Nielsen describes as recognition versus recall [3]: a blind user has to listen to each item in a list, which is slow and tiring. Raman makes the same point about Emacspeak: a speech interface has to do more filtering so the user does less listening [4].

The startup interface solves a separate problem. When HEAT first opens there is no keyboard entry point. We solved this by intercepting key events globally using `KeyboardFocusManager.addKeyEventDispatcher()` before the main window is ready. Normal Swing key bindings do not work at this point because no window has focus, so the global dispatcher was the only option.

B. Software Design

New classes follow the existing HEAT package structure. Accessibility utilities are in a new accessibility package: `TTSManager`, `ErrorSimplifier`, `ColorThemeManager`, `FontSizeManager`, `KeyboardGuide`. UI additions follow existing conventions: panels in `view/panels`, windows in `view/windows`, dialogs in `view/dialogs`.

Dependencies in the accessibility package flow in one direction for the purely utility classes (`ErrorSimplifier` and `ColorThemeManager` have no external dependencies; `TTSManager` and `KeyboardGuide` depend on `SettingsManager`). `FontSizeManager` is an exception: because it must apply size changes immediately to the editor and console components, it depends on `WindowManager` as well as `SettingsManager`. This dependency is an architectural trade-off between keeping the accessibility package independent and avoiding the alternative—having `WindowManager` poll `FontSizeManager` on every render cycle.

C. Colour Design

Deuteranopia colours were taken from the Wong (2011) accessible colour palette [5]: amber #E69F00 for error text and blue #0072B2 for prompt text. Both were verified against Coblis [6] before submission. Screenshots showing the before-and-after colour appearance under Deuteranopia simulation are in the GitLab wiki.

D. UML Diagrams

PlantUML class, sequence, and activity diagrams grounded in the actual source code are at `heat/documentation/` in the repository. The class diagram shows the structure of the accessibility extension. The TTS sequence diagram traces the path from GHCi output to spoken speech. Activity diagrams cover the splash screen, `KeyboardOpenFileDialog`, `KeyboardNewFileDialog`, and `KeyboardFileExplorerDialog` interaction flows. The colour blind mode and font size sequence diagrams show the toggle and increase/decrease flows respectively.

VI. IMPLEMENTATION

A. TTS (Yarushah)

`TTSManager.java` is a singleton. It maintains a `BlockingQueue` of strings and a background thread that

drains the queue, running the OS speech command for each message in order. Using a queue rather than direct execution prevents messages from interrupting each other—a producer-consumer concurrency pattern.

The startup delay uses a boolean flag (`startupComplete`) set to false on construction. `Main.java` calls `setStartupComplete()` after a 2-second sleep following `wm.setVisible()`. Until the flag is set, `speak()` calls are silently dropped. This prevents GHCi’s startup status transitions from triggering spurious speech during application launch.

`ErrorSimplifier.java` uses a series of `String.contains()` checks against 10 known GHCi error patterns. Unknown errors fall through to a raw-line fallback so no error is silently swallowed. The 10 patterns are: no instance for, not in scope, parse error, couldn’t match type, ambiguous type, infinite type, non-exhaustive patterns, variable not in scope, failed modules loaded, and ok modules loaded.

B. Colour Blind Mode (Thabitha)

`ColorThemeManager.java` defines two static inner classes, `DefaultTheme` and `DeuteranopiaTheme`, each containing static final `Color` fields (`ERROR_COLOR`, `PROMPT_COLOR`). A static boolean selects which set of constants is returned by `getErrorColor()` and `getPromptColor()`. The toggle is instantaneous with no file I/O overhead. `refreshStyles()` re-walks the console document and re-applies the current theme colours to all existing text immediately. `Cmd+Shift+C` fires `ToggleColourBlindAction`, which flips the flag, saves to `SettingsManager`, calls `refreshStyles()`, and speaks a TTS confirmation.

C. Font Size Control (Thabitha)

`FontSizeManager.java` uses static methods to keep font state centralised. `increaseFontSize()` and `decreaseFontSize()` clamp to the 8pt to 36pt range in 2pt steps and immediately update both the editor and console components. The current size is written to `heat.settings` on each change so it persists across restarts.

`AccessibilityPanel.java` registers a `ChangeListener` on the `JSlider` and `ActionListeners` on the `A+` and `A-` buttons. All three paths call the same `FontSizeManager` methods, ensuring the slider and buttons cannot show different values.

Keyboard shortcuts implemented on customer feedback. Following the customer interaction with Dr. Rogério de Lemos on 27 May 2026, the team implemented `Cmd+=` and `Cmd+-` keyboard shortcuts for font size increase and decrease respectively. The shortcuts are registered in `ActionManager.java`

as `IncreaseFontSizeAction` and `DecreaseFontSizeAction` and are available globally throughout the application. The keyboard guide (F1) announces both shortcuts so a blind user can discover them without reading the interface.

Dialog proportional scaling. Font size changes made through `Cmd+=` or `Cmd+-` are also propagated to the three keyboard file dialogs, so that dialog text fields and result lists scale proportionally with the user’s chosen size.

D. Accessible Startup Interface (Goutham, TTS by Yarushah)

The original HEAT had no keyboard entry point before the main window was fully loaded. `SplashWindow.java` addresses this. It is an undecorated `JFrame` displayed before the main window becomes visible. A `KeyEventDispatcher` is registered with the global `KeyboardFocusManager`, which intercepts key events regardless of which component has focus. This was necessary because an undecorated `JFrame` with no focused child components receives no keyboard input through the normal Swing event chain.

Yarushah wired TTS into the startup interface so that pressing `Cmd+N` speaks “Creating new Haskell file.” before the dialog opens. Pressing `Escape` calls `TTSManager.stopSpeaking()` to cut any in-progress speech immediately. The startup interface is dismissed by user action (`Cmd+N`, `Cmd+O`, or `Escape`) or automatically when `WindowManager.dismissSplash()` is called on a successful compilation.

E. Keyboard File Navigation (Goutham, TTS by Yarushah)

Three dialogs cover the file operations a student needs: opening an existing file (`KeyboardOpenFileDialog`), creating a new file (`KeyboardNewFileDialog`), and moving or copying a file (`KeyboardFileExplorerDialog`). All three are keyboard-only.

`KeyboardOpenFileDialog` shows a search field and a scrollable results list. The user types a file-name fragment, presses `Enter`, and navigates the results with arrow keys. Search results trigger a spoken count. `Cmd+F5` reads the full absolute path aloud. `KeyboardNewFileDialog` has two fields (filename and directory) with `Tab` navigation and automatic directory creation. `KeyboardFileExplorerDialog` uses `CardLayout` to present a four-step workflow: search, action selection, move-source confirmation, and destination entry.

All three dialogs set `getAccessibleContext().setAccessibleContext()` on every interactive component for screen reader compatibility.

F. Settings Integration (Yarushah, absorbed from Rinku)

`Settings.java` adds `TTS_ENABLED`, `HIGH_CONTRAST_MODE`, and `DEUTERANOPIA_MODE` as public static final String constants. Using named constants means a typo in a key name is caught at compile time rather than silently producing a null at runtime. `SettingsManager.java` adds all three new keys to `createDefaultProperties()` so sensible defaults (TTS on, both colour modes off) are written on first run. This work was originally assigned to Rinku Kumari but absorbed into the final merge by Yarushah after Rinku's code could not be integrated.

VII. QUALITY ASSURANCE

A. Testing Approach

Acceptance criteria testing was the primary QA method. Each user story has between one and four criteria defined in Given/When/Then format. 56 criteria were defined; 50 pass. AC-12 (TTS disabled mode) was tested by setting `TTS_ENABLED=false` in `heat.settings`: HEAT confirmed completely silent. Status: Pass.

Unit testing was applied to the pure logic classes with no Swing dependencies. Four unit test files were written targeting `ErrorSimplifier`, `ColorThemeManager`, `FontSizeManager`, and `KeyboardDialogsTest`. These 51 tests can be run without launching the application. Manual acceptance testing was used for GUI and audio behaviour where automated testing was not cost-effective.

The functional test plan (`test/functional/FunctionalTestPlan.md`) documents 16 manual black-box scenarios covering all features. Fourteen pass. FT-10 and FT-11 (font size keyboard shortcuts) were tested using the A+ and A- buttons; the shortcut binding in the editor context is a known limitation documented in AC-36.

B. Cross-Feature Testing

Features were tested by a team member other than the implementer where possible. TTS acceptance tests were run by Goutham. Font size persistence across restarts was verified by Yarushah. Splash screen keyboard behaviour was verified by Thabitha. For the colour blind mode, Coblis [6] was used to generate Deuteranopia simulation screenshots before and after the colour change. These screenshots confirm that the amber and blue replacement colours are distinguishable where the original red and green are not. The screenshots are in the GitLab wiki.

C. Code Inspection

Pre-merge inspection confirmed: `BlockingQueue` thread safety in `TTSManager`; no hardcoded colours remaining in `ConsoleWindow`; `ErrorSimplifier` pattern ordering correct (specific before general); all settings

keys referenced via named constants throughout the codebase; all `getAccessibleContext()` calls null-guarded following a `NullPointerException` encountered in testing.

D. Confidence

We have high confidence in TTS (tested against all 10 error patterns and all five status transitions) and in the colour blind mode (colours from a peer-reviewed palette, verified with a simulator). We have moderate confidence in the keyboard file dialogs: testing was performed primarily on macOS and behaviour on Windows and Linux has not been verified.

VIII. TOOLS

All four members pushed commits to the repository at <https://git.cs.kent.ac.uk/comp7024/g5>. GitLab issues #2 and #3 tracked TTS and keyboard navigation work. All development was done in Visual Studio Code with the Java extension pack. GHCi 9.6.7 was installed via GHCCup for local testing on macOS. Coblis [6] was used to simulate Deuteranopia on HEAT screenshots before and after the colour theme change. macOS VoiceOver was used to verify accessible names on toolbar buttons.

Claude Code (Anthropic, Claude Sonnet 4.6) was used in four ways, all disclosed per the module AI use policy: (1) initial codebase navigation and setup; (2) TTS debugging, identifying the startup noise issue and suggesting the `startupComplete` flag; (3) local merge conflict resolution between Goutham's codebase snapshot and Yarushah's incremental commits; (4) generating the initial implementation of `KeyboardFileExplorerDialog.java` (490 lines) as part of commit e160f98. All other feature code was written by team members.

IX. UNDERSTANDING AND INSIGHT

A. What Went Well

TTS was the hardest feature to build and we finished it first. That turned out to be useful in a way we did not expect: the other team members could use TTS to hear when their own features were working. Using OS-native speech was the right call—the system commands just worked on macOS immediately with no configuration. The `BlockingQueue` design for TTS was something we got right after getting it wrong: the first version cancelled any in-progress speech every time a new message arrived, producing truncated announcements during compilation. Switching to a queue so messages play in order fixed it.

B. What Did Not Go Well

The first two days were slower than expected due to unfamiliarity with multi-user GitLab workflows. A merge conflict arose because Goutham pushed a complete codebase snapshot while Yarushah had incremental feature commits

on top of a different base. We resolved the conflict locally with Claude Code’s help, which was also where we first properly understood what a merge conflict actually is.

Rinku pushed non-functional code directly to the master branch without consulting the group and without the mandatory code review agreed at the kickoff meeting. This introduced broken code into the shared codebase, blocked other members from building, and disrupted the team’s ability to push and pull reliably. As a result the team stopped relying on GitLab for sharing work in progress and instead shared files locally, working on them independently and only pushing once features were complete. This is why the final merge was a manual integration of three separate codebases rather than a series of incremental branch merges. Rinku’s assigned features were not delivered, requiring Thabitha to take on additional work in the final two days.

C. Lessons Learned

The biggest lesson was the difference between using Git alone and using Git as a team: small commits on named branches, frequent merges, and file ownership agreed before work starts. We would start integration testing earlier and establish a mid-week check-in requiring each person to demonstrate working code. Testing with an actual visually impaired user was the most significant gap in our process.

X. FUTURE WORK

Each item below comes from a specific limitation we hit.

- 1) **Java Accessibility API compliance.** Set accessible roles, states, and relations on all interactive components. Test with VoiceOver and NVDA.
- 2) **Continuous TTS guidance in KeyboardFileExplorerDialog.** Speak available options at each step, confirm disk operation outcomes aloud, and narrate validation errors inline.
- 3) **Fuzzy search with recency weighting.** Files opened recently ranked higher to reduce typing.
- 4) **Non-speech audio earcons.** Distinct tones for compile success and failure before spoken detail [4].
- 5) **Undo and audit trail for file operations.** Spoken confirmation and operation log for destructive disk changes.
- 6) **Evaluation with a visually impaired user.** The most important validation step—should be done before any other item.
- 7) **Visual accessibility inside keyboard dialogs.** Wire `FontSizeManager` and `ColorThemeManager` into all three dialogs.

XI. CONCLUSIONS

We extended HEAT with five accessibility features targeting the barriers we identified. A blind student can now hear every status change and error in plain English. A student with Deuteranopia can distinguish error text from prompt

text. A partially sighted student can increase the font without opening a menu. Any keyboard-only user can create a file, open a file, and navigate the file system from the moment the application starts, without a mouse.

The feature we are most satisfied with technically is the startup interface, because it solves a problem that the other dialogs could not: giving a keyboard entry point before the main window exists. The design idea—that blind users are better served by typing a name to search than by browsing a visual list—informed all three file dialogs and is the thread that connects the keyboard navigation work together.

ACKNOWLEDGMENT

We thank Rogério de Lemos and the project consultants for guidance during project week.

REFERENCES

- [1] University of Kent, Department of Computing, *HEAT – Haskell Educational Advancement Tool*. Available: <https://git.cs.kent.ac.uk/comp7024>
- [2] Sun Microsystems Laboratories, *FreeTTS: A Speech Synthesis System Written Entirely in the Java Programming Language*, 2003. Available: <https://freetts.sourceforge.net/>
- [3] J. Nielsen, *Usability Engineering*. Morgan Kaufmann, 1993.
- [4] T. V. Raman, *Auditory User Interfaces: Toward the Speaking Computer*. Kluwer Academic Publishers, 1997.
- [5] B. Wong, “Points of view: Color blindness,” *Nature Methods*, vol. 8, no. 6, p. 441, 2011.
- [6] Coblis Colour Blindness Simulator. Available: <https://www.color-blindness.com/coblis-color-blindness-simulator/> [Accessed 3 Jun. 2026]
- [7] V. Potluri et al., “CodeTalk: Improving programming environment accessibility for visually impaired developers,” in *Proc. CHI 2018*, ACM, 2018.
- [8] C. M. Baker, L. R. Milne, and R. E. Ladner, “StructJumper: A tool to help blind programmers navigate and understand the structure of code,” in *Proc. CHI 2015*, ACM, 2015.
- [9] A. Stefik and S. Siebert, “An empirical investigation into programming language syntax,” *ACM Trans. Comput. Educ.*, vol. 13, no. 4, 2013.
- [10] World Wide Web Consortium, *Web Content Accessibility Guidelines (WCAG) 2.1*, W3C Recommendation, June 2018. Available: <https://www.w3.org/TR/WCAG21/>

APPENDIX

APPENDIX: UML DIAGRAMS

The PlantUML source files are at `heat/documentation/` in the repository and can be rendered with PlantUML or the PlantUML VS Code extension:

- `diagram_class.puml` — Class diagram of the accessibility extension
- `diagram_sequence.puml` — TTS path from GHCi output to speech
- `diagram_colourblind.puml` — Deuteranopia colour blind mode toggle sequence
- `diagram_fontsize.puml` — Font size increase/decrease sequence
- `diagram_startup.puml` — Accessible startup interface activity diagram
- `diagram_activity.puml` — KeyboardOpenFileDialog interaction flow
- `diagram_newfile.puml` — KeyboardNewFileDialog interaction flow
- `diagram_fileexplorer.puml` — KeyboardFileDialog 4-panel flow